Capstone Report: Low-Latency Order Book

Name: Apurv Dugar
Roll Number: 24BCS10107
Email: apurv.24bcs10107@sst.scaler.com

1. Results Table

The benchmark workload was executed on a Linux system with **1,000,000 deterministic operations** (70% Add, 20% Cancel, 10% BBO query, plus depth queries every 100 ops; seed 42). Timings represent the **median of 5 runs**. Hardware counters were recorded using Linux perf stat.

Metric	Implementation A (std::map)	Implementation B (priority_queue)	Implementation C (Fixed Array)
Total time (ms)	246.90 ms	587.31 ms	116.63 ms (Fastest)
ns / add	222.56 ns	139.83 ns	80.56 ns (Fastest)
ns / cancel	343.72 ns	252.48 ns	192.26 ns (Fastest)
ns / BBO	15.28 ns (Fastest)	40.33 ns	15.43 ns
IPC (Instructions / Cycle)	3,793,325,498 / 4,896,644,280 = 0.77	18,056,324,461 / 11,443,971,607 = 1.58 (Best)	3,291,248,550 / 2,674,611,528 = 1.23
Branch misses (%)	42,584,856 / 757,570,118 * 100 = 5.62%	83,694,766 / 2,549,091,916 * 100 = 3.28%	8,839,619 / 622,534,233 * 100 = 1.42% (Best)
L1-dcache-load-misses	72,095,131 (72.10 M)	157,265,872 (157.27 M)	27,911,605 (27.91 M) (Lowest)
Cache misses (LLC)	28,514,101 (28.51 M)	30,451,589 (30.45 M)	22,592,790 (22.59 M) (Lowest)
Page faults	22,642	62,543 (Highest)	22,503 (Lowest)

2. Performance Analysis

Which implementation is fastest?

add_order: Implementation C (Fixed Array) is fastest at 80.56 ns
cancel_order: Implementation C (Fixed Array) is fastest at 192.26 ns
get_bbo: Implementation A (std::map) and Implementation C (Fixed Array) are tied at ~15.3–15.4 ns
Overall Runtime: Implementation C finishes the entire 1,000,000 operation workload in 116.63 ms, making it 2.1× faster than Impl A (246.90 ms) and 5.0× faster than Impl B (587.31 ms).

Explanation -

- Direct O(1) Array Indexing: Adding an order in Implementation C simply computes an offset (p - MIN_BID) and does bids[offset] += qty. It does not perform any dynamic heap allocation, tree rebalancing, or heap re-ordering.
- Compact Contiguous Memory: The 5,000 price levels for bids and 5,000 price levels for asks are flat contiguous arrays (5000 * sizeof(long) = 40 KB each). The entire price table fits directly inside the CPU's fast L1/L2 caches, so memory reads and writes happen almost instantly.

Branch Prediction

Most branch misses: Impl A (std::map) has the highest branch misprediction rate at 5.62% (42.58M branch misses).
Highest raw misses: Implementation B has 83.69M branch misses, because it executes an enormous total of 2.55B branches during heap reorganizations.
Best branch prediction: Implementation C has only 1.42% branch misses (8.84M misses out of 622.53M branches).

Explanation -
In Impl A (std::map): An RB-tree traverses left and right child pointers at each tree node, because order prices are randomly distributed between 50.00 and 150.00, whether the next branch goes left or right has an equal probability.
In Impl B (priority_queue): Shifting heap elements repeatedly compares parent and child elements across arbitrary heap locations.

In Impl C (fixed sized array): Impl C avoids searching altogether by calculating the array index directly with arithmetic (p - MIN_BID). When an order arrives, it updates the best price with: if (p > best_bid) best_bid = p;
Since best_bid quickly reaches the top of the price range (near 99.99), incoming random orders rarely exceed it. The CPU branch predictor quickly learns that this branch is almost never taken so, it makes the correct prediction in most cases.

Cache Behavior

Most cache misses: Impl B has the worst cache behavior with 157.27M L1 data-cache load misses & 30.45M LLC misses.
Best cache behavior: Impl C has only 27.91M L1-dcache misses (2.6× fewer than A, 5.6× fewer than B) and 22.59M LLC misses.

Explanation -
Impl A (std::map): Every distinct price level allocates a separate RB-tree node on the heap. Traversal requires chasing pointers (node->left, node->right, node->parent) across non-contiguous heap addresses. The CPU cannot prefetch these scattered addresses into cache.
Impl B (priority_queue): Backed by a contiguous std::vector, but because of lazy deletion, the vector expands to hold hundreds of thousands of elements.
Heap navigation hops between index i and children 2i+1, 2i+2. When the heap is huge, there are cache level misses.
Impl C: Uses two flat arrays of 5,000 elements each (long bids[5000], long asks[5000]). Total memory is only 10,000 * 8 bytes = 80 KB. The entire price structure easily fits inside the CPU's L1/L2 caches. Contiguous array elements are automatically loaded by hardware prefetchers, leading to minimal cache misses and smooth execution.

Page Faults

Highest page faults: Implementation B caused 62,543 page faults, which is nearly 3× higher than Implementation A (22,642) and Implementation C (22,503).

Explanation -
In Implementation B: Because cancelled orders are never removed from the priority queue until they reach the top, the heap vector constantly grows throughout the 1,000,000 operations. This forces repeated vector reallocations and new physical memory pages from the Linux kernel.
Implementation C allocates zero additional pages during price-level updates.

Production Choice

If I were deploying only one implementation to a live, production high-frequency trading (HFT) system, I would choose Implementation C (Fixed Array).

Explanation -

- Latency: It delivers the fastest latency: 80.56 ns for add_order, 192.26 ns for cancel_order, and 15.43 ns for get_bbo. It processes 1,000,000 market operations in just 116.63 ms.
- Predictability: Because add_order simply increments an array index, it performs zero dynamic heap allocations (malloc/free). Unlike std::map (which allocates on new price levels and rebalances trees) or priority_queue (which reallocates vector buffers).
- Memory Usage: The price level storage is completely static and bounded at compile time: exactly 80 KB for 10,000 ticks. It cannot suffer from memory leaks, unbounded growth, or heap fragmentation.
- The only limitation of impl C is scanning backwards/forwards when the best price level is completely cancelled to zero. However, in real markets, prices are usually at the top of the book. Linear array scans are fast on modern CPUs because hardware prefetches sequential memory lines into cache before the instruction even requests them.

What Would You Try Next?

- Array based orders registry instead of std::unordered_map: Instead of using an unordered map we can use an array if the order_ids are sequential.
- Object Pooling / Pre-allocation: Pre-allocate all memory structures at application startup so that the critical path executes with no context witches and no malloc calls.
- Cache Line Alignment (alignas(64)): Align arrays and cache-sensitive variables (such as best_bid and best_ask) to 64-byte boundaries using alignas(64) to ensure optimal CPU cache line utilization and avoid false sharing in multi-threaded environments.
